Deliverable report

"Alessio Blascovich": Mat: 257561, alessio.blascovich@studenti.unitn.it, GitRepo:
https://github.com/elblasco/GPU-Computing-2025-257561

***Abstract*—The sparse matrix-dense vector multiplication (SpMV) is an algebraic operation in which a sparse matrix is multiplied by a dense vector. SpMV is widely used in real-world applications such as scientific and engineering computing, data mining, and graph analysis.**

This deliverable discusses the implementation and the performance analysis of a series of SpMV algorithms for sparese matrices encoded using the COOrdinate (COO) format. These algorithms are implemented for *CUDA* capable GPU. Regarding the GPU implementations, the main techniques used leverage on warp instructions or shared memory optimisations.

***Index Terms*—Sparse Matrix, SpMV, *CUDA*, Parallelization, Storage Format**

I. Introduction

SpMV is considered a fundamental algebraic operation in many scientific fields such as computational fluid dynamics, optimisation and graph algorithms.

The main challenges in implementing an efficient SpMV algorithm differ in CPU and GPU, and are strictly bound to the type of matrix representation used COO, Compressed Sparse Row (CSR), ELLpack-Itpack (ELL),.... For the purpose of this deliverable only COO representation is taken into account.

From the CPU point of view, the main issue is to maximise the cache hits throughout the execution, data consistency is not a problem since only single core CPUs were used.

On the GPU side the problem of maximising cache hits remains, and on top of that, race conditions are added. Since in COO format the length of a row is not specified, multiple threads could be reading different portions of the same row and therefore, writing on the same output memory area.

II. Problem Statement

The SpMV operation is formally defined in Equation (1), where $\boldsymbol{A}$ is a $n \times m$ sparse matrix, $\boldsymbol{X}$ a $m \times 1$ dense vector and $\boldsymbol{Y}$ another $n \times 1$ dense vector.

$$\boldsymbol{Y} = \boldsymbol{A} \cdot \boldsymbol{X} \tag{1}$$

A. Storage Format

The COO format is a representation for sparse matrices. COO uses 3 different arrays to store al the necessary information. One array is dedicated to store all the values, while the other 2 contain the corresponding row and column indices, ideally the arrays are sorted first by row index and then by columns index. An example of a COO encoding is shown in Figure 1.

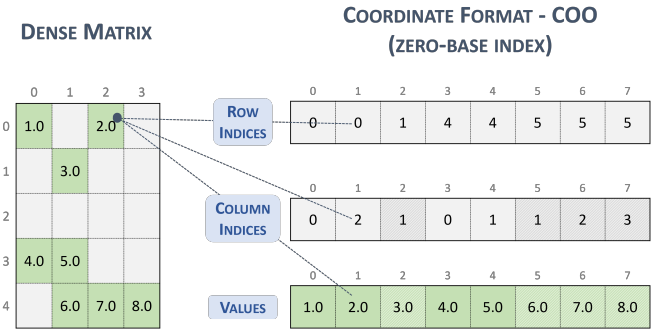

Fig. 1. Encoding of a matrix in COO format

B. Parallelization

Three different kernels have been used for the *CUDA* capable GPU. Although they differ by the kind of memory and optimisations used, all of them leverage on the fact that the COO arrays are sorted first by row and then by column. In general the multiplication problem has been reduced to a prefix sum computation. To optimise the prefix sum computations I tried to minimise writes on global memory by using warp level primitives and the shared memory.

III. State of the Art

Starting from the implementations presented in this deliverable, performances could be drastically enhanced. Fukaya et al. [1], despite focussing on the HDC storage format, improved the SpMV CPU based by exploiting diagonalization patterns, SIMD instructions and *OpenMP* directives. On the GPU side Chu et al. [2] used the CSR format and a more sophisticated load balancing to improve the performances. NVIDIA itself made available a set of libraries to manage and use COO formatted matrices, namely *cuSPARSE*.

In the NVIDIA library *cuSPARSE*, and many others commercial libraries, the problem is redefined according to a more complex equation state in Equation (2).

$$\boldsymbol{Y} = \alpha op(\boldsymbol{A}) \cdot \boldsymbol{X} + \beta \boldsymbol{Y} \tag{2}$$

With α and β being two scalars, and *op* being a matrix transformation such as a transposition.

IV. Methodology and Contributions

The test were carried out on a remote cluster. I compared all the kernels against a set of matrices, for every kernel I executed multiple run to collect more meaningful data.

All the algorithms assumed that the resulting array is empty, so I explicitly initialised every element of it to the value 0.

Hereafter I will refer to the best kernel presented in the previous deliverable as "baseline".

A. Algorithms

For the GPU implementation I have developed three kernels, they are divided into two main categories. Kernel that leverage on shared memory and kernels that leverage on warp level primitives. Algorithm 1 was the first kernel developed using warp-level primitives. Every thread proceed with a striding of $numThreads$ values, initially every thread computes the product between its the matrix value and the value for the corresponding dense array. This values are then combined using a prefix-sum based on warp-level instructions.

Algorithm 1 SpMV algorithm with WARP level primitives

Require: The input vectors val, row and col of size nnz, array arr of size $columns$ and an array res of size $rows$.

procedure FUNCTION(row, col, val, arr, res, ...)
  $start \leftarrow threadId$
  $pos \leftarrow threadId\&(WARP\_SIZE - 1)$
  $total \leftarrow gridDim \cdot blockDim$
  for i in $\{start \dots nnz\}$ with $i = i + total$ **do**
    $row\_id \leftarrow row[i]$
    $col\_id \leftarrow col[i]$
    $val\_id \leftarrow val[i]$
    $PREFIX\_SUM\_WARP(\dots)$
    $row\_next \leftarrow shfl\_down(\texttt{0xff}\dots, row, 1)$
    if edge detected **then**
      `atomicAdd(&res[row], prod);`
    end if
  end for
end procedure

Algorithm 2 Prefix-sum with WARP level primitives

Require: The values for $product$, row and pos.

procedure PREFIX_SUM_WARP($product, row, pos$)
  for x in $\{1 \dots WARP\_SIZE\}$ with $x = x \cdot 2$ **do**
    $prev\_row \leftarrow shfl\_up(\dots, row, x)$
    $prev\_prod \leftarrow shfl\_up(\dots, product, x)$
    if $pos \geq delta \wedge prev\_row = row$ **then**
      $product = product + rev\_prod$
    end if
  end for
end procedure

As mention in Section II-B I reduced the problem to a prefix sum, as a matter of fact Algorithm 2 exploits warp-level primitives to compute a prefix sum. Algorithm 2 is more refined w.r.t. simple prefix sum, since it has to take into account the possibility that within the same warp we could be summing different rows. In Figure 2 it is described a possible execution for the prefix sum procedure, in red the values that are going to pass the edge detection check and are going to be written in global memory. The next improvement introduced was the loop unroll of the main loop in Algorithm 2, using compiler directives. To be precise, I used `#pragma unroll`, since the loop always iterates in an interval known at compile time, in fact `WARP_SIZE` is a macro defined as the value 32.

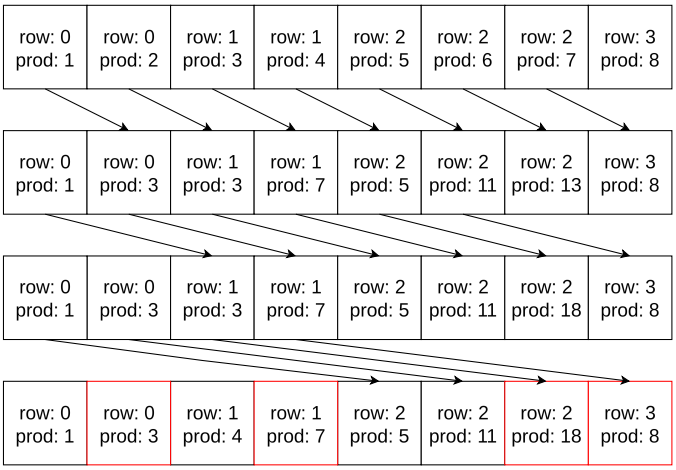

Fig. 2. Execution of prefix sum with warp size 8

I also tried to improve the baseline proposed in deliverable 1 by using dynamically allocated shared memory. I used a similar technique to the one described in Algorithm 1. Initially the values are copied into the shared memory then an algorithm similar to Algorithm 2 and pictured in Figure 2.

V. System Description and Experimental Set-up

A. System Description

In this section, I will also report the version of piece of software that I did not use. I was not able to use *ncu* since the node in which I was working crashed every time I was trying to use it.

For this deliverable I could not use my own machine since it does not have a *CUDA* capable GPU, so I only used the remote cluster. Table I describes all the software used, while Table II describe the hardware that I used.

Software	Version	Git Branch	Git Hash
`GCC`	11.5.0		
`CUDA`	12.5.0		
`nsys`	2024.2.3.38		
`ncu`	2024.2.0.0		
`SbatchMan`		example	40d869a
`distributed_mmio`		main	3563a2f

TABLE I
SYSTEMS SOFTWARES

Regarding the flags used:

- `GCC: -O3 -g`
- `CUDA: --gpu-architecture=sm_80 -m64 -O3 -g --compiler-options "-Wall -Wextra" -Iinclude -lcusparse -lineinf`

Algorithm 3 Prefix-sum with WARP level primitives

Require: The input vectors val, row and col of size nnz, array arr of size $columns$ and an array res of size $rows$.

procedure FUNCTION(row, col, val, arr, res, ...)
  $start \leftarrow threadId$
  $pos \leftarrow threadId\&(WARP\_SIZE - 1)$
  $total \leftarrow gridDim \cdot blockDim$
  $shared \leftarrow \_\_shared\_\_$
  for i in $\{start \dots nnz\}$ with $i = i + total$ **do**
    $row\_id \leftarrow row[i]$
    $col\_id \leftarrow col[i]$
    $val\_id \leftarrow val[i]$
    $val\_id \leftarrow 1$
    $shared[start] \leftarrow val\_id * arr[col\_id]$
    for d in $\{blockDim.x - 1 \dots 0\}$ with $d = d/2$ **do**
      __syncthreads
      $bi \leftarrow offset + start$
      $other\_row \leftarrow row[i + bi]$
      if $start < d \wedge row\_id = other\_row$ **then**
        $shared[bi] \leftarrow shared[bi] + shared[start]$
      end if
      $offset = offset \cdot 2$
    end for
    __syncthreads
    $row\_next \leftarrow shfl\_down(\texttt{0xff} \dots, row, 1)$
    if edge detected **then**
      `atomicAdd(&res[row_id], shared[start]);`
    end if
  end for
end procedure

Processor	Cores per Socket	RAM	GPU
AMD EPYC 9334	32 at 2 GHz	771 GiB	NVidia L40s

TABLE II
SYSTEMS DETAILS

B. Dataset description

To add a bit more of generality to the dataset I picked six matrix from six different domains, from the site https://sparse.tamu.edu/. Since all the matrices are squared, I decided to describe their sizes by writing only rows. I had to avoid rectangular matrices since they were a minority w.r.t to the number of square matrices present on the site. I tried to balance different domain of application with different sparsity ratio.

This is the list of all domains of applications which I picked from:

1) Electromagnetics Problem;
2) Undirected Weighted Graph;
3) Structural Problem;
4) 2D/3D Problem;
5) Optimisation Problem;
6) Undirected Graph.

Table III summarises all the matrices used.

Name	Rows	Pattern Entries	Sparsity
`dielFilterV3real`	1,102,824	89,306,020	7e−5
`Flan_1565`	1,564,794	117,406,044	5e−5
`mawi_201512020330`	226,196,185	480,047,894	9e−9
`mycielskian19`	393,215	903,194,710	6e−3
`nlpkkt160`	8,345,600	229,518,112	3e−6
`Queen_4147`	4,147,110	329,499,284	2e−5

TABLE III
INPUT MATRICES

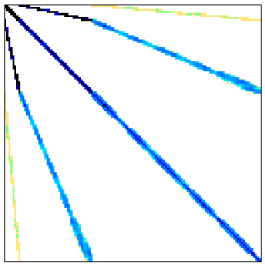

Fig. 3. `dielFilterV3real`

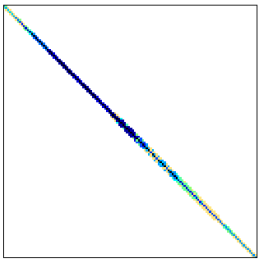

Fig. 4. `Flan_1565`

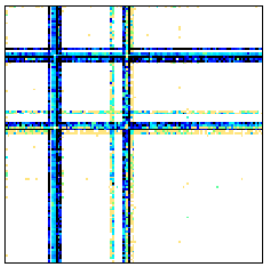

Fig. 5. `mawi_201512020330`

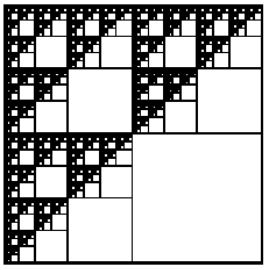

Fig. 6. `mycielskian19`

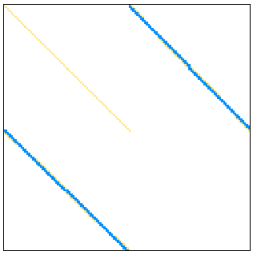

Fig. 7. `nlpkkt160`

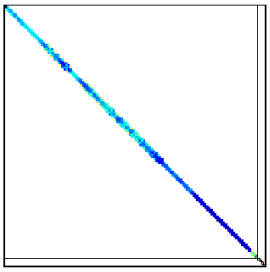

Fig. 8. `Queen_4147`

C. Experimental Set-up

For every matrix I measured a set of metrics using an approach similar to the first deliverable. For every matrix I spawned a job which ran multiple times every kernel and gathered info about the performances. All the metrics were averaged used the geometric mean and were: average execution time, average peak GFLOP/s developed and average bandwidth used. I was able to test all the algorithms in a single

job since I switched to the node *edu02*, which is equipped with the more powerful NVidia L40s, w.r.t. *edu01* which is equipped with NVidia A30.

Every job carried out its measurements performing multiple runs of the same kernel. The kernel tests were split into two period, a warm-up period and a "valid" period, the data has been collected only during the "valid" period. The distinction were necessary in order to war-up the GPU caches. I decided to keep this warm-up cycles even though a later analysis with *nsys* highlighted the fact that even after a single ran the execution time stabilises.

To automatise the matrix parsing, and their conversion in binary format, I used a fork of `distributed_mmio`. Me and my colleague *Matteo Di Noia* forked the original repo to fix a bug in the binary conversion of symmetric matrices. The repository is available on his *GitHUb profile*.

The job submission were managed by *SbatchMan*, this automation allowed me to create multiple experiments and submit all of them. In particular I created an experiment for normal metrics gathering and an experiment for profiling via *nsys* and *ncu*; to both profilers *nvtx* was added to increment the readability.

VI. Experimental Results

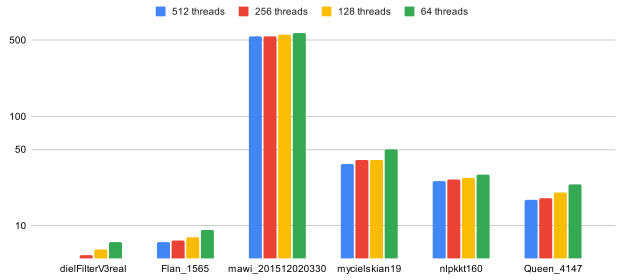

Fig. 9. Execution time (ms) for the baseline kernel (lower is better)

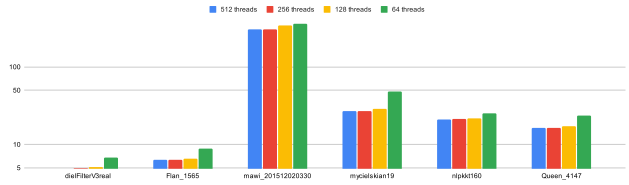

Fig. 10. Execution time (ms) for the warp level kernel (lower is better)

As mentioned in Section V-A *ncu* caused several problems in the cluster so my analysis is restricted only to the metrics computed using *CUDA* events to keep track of time. Even *nsys* has not been very helpful since it does not provide this kind of insights about the kernel performances.

I experimented for all the kernels with various sizes of grid a block, however for the sake of brevity I did not print the GFLOP/s and GB/s charts for all the kernels. Overall the charts for the GFLOP/s developed respect the same evolution

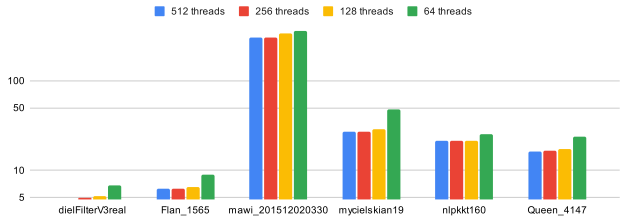

Fig. 11. Execution time (ms) for the warp level kernel with unroll (lower is better)

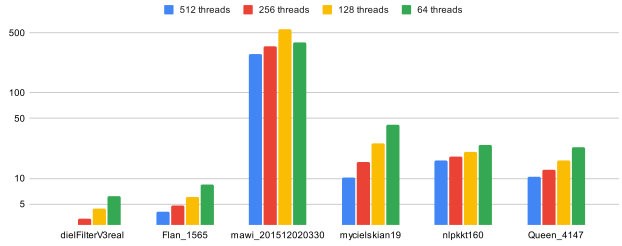

Fig. 12. Execution time (ms) for the share memory kernel (lower is better)

of Figures 9 to 12. As can be seen the best performances are yielded by block size between 512 and 256 threads. This is because if we use less thread per block these fewer threads will be overloaded with an excessive workload, that can be parallelised using the strategies described in Section IV-A.

Figures 9 to 12 highlight how the kernel with the fastest execution time is, on average, the kernel which exploit shared memory. However this is not always the case since it perform lesser read and write on the global memory it has a lower bandwidth, as showed in Figures 14 and 15.

Despite not having access to *ncu* I gained some intuitions regarding possible optimisations for the algorithms presented. First and foremost in Algorithm 1 the access pattern can be drastically improved by assigning to every block (and by extension to every warp) the next adjacent chunk of the matrix.

Following the NVIDIA's white-paper for the L40s, I drew the roofline model for single precision floating point, since in my code I used `float` as data type to represent values within the matrix and vectors. Figures 16 and 17 highlights the fact that Algorithm 1 with unroll and Algorithm 3 are indeed a memory bounded.

VII. Conclusions

Without the help of *ncu* has been difficult trying to extrapolate useful insights and improve the kernels. However even without these tools I am sure that exist better solutions. An example is the algorithm proposed by Belloch [3], which can be easily extended to array (COO encoded matrices), which sizes are not power of two. Although these kind of algorithms has to be modify in order to take into account the case in which two element can not be summed together or the case in which the result is not at the edges of a block/warp.

The test phase can also be improved as well, validation phase can be introduced. In this phase the results produced

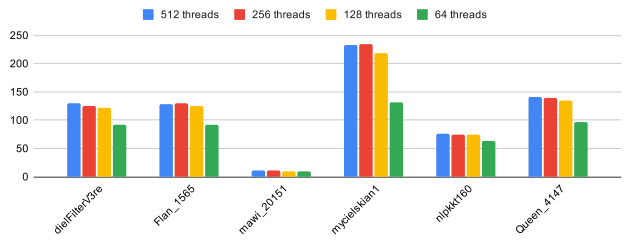

Fig. 13. Average GFLOP/s developed by the warp level kernel with unroll (higher is better)

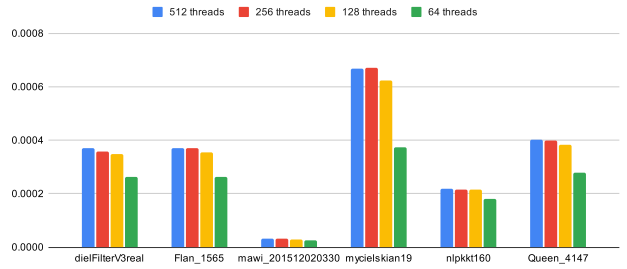

Fig. 14. Average GB/s developed by the warp level kernel with unroll (higher is better)

by an experimental kernel are compared against the results generated by a well know kernel to verify the correctness.

Also, the comparison with the, currently in use, *cuSPARSE* algorithms could represents a valuable benchmark metrics for the kernel created and presented.

References

[1] T. Fukaya, K. Ishida, A. Miura, T. Iwashita, and H. Nakashima, "Accelerating the spmv kernel on standard cpus by exploiting the partially diagonal structures," 2021. [Online]. Available: https://arxiv.org/abs/2105.04937

[2] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing spmv on gpu," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 115–128. [Online]. Available: https://doi.org/10.1145/3588195.3593002

[3] G. Blelloch, *Vector Models for Data-parallel Computing*, ser. AI Series. MIT Press, 1990. [Online]. Available: https://books.google.it/books?id=wN8mAAAAMAAJ

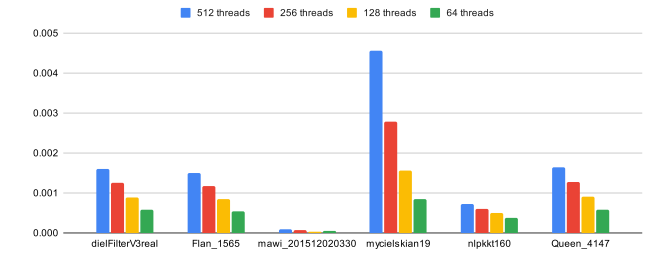

Fig. 15. Average GB/s developed by the share memory kernel (higher is better)

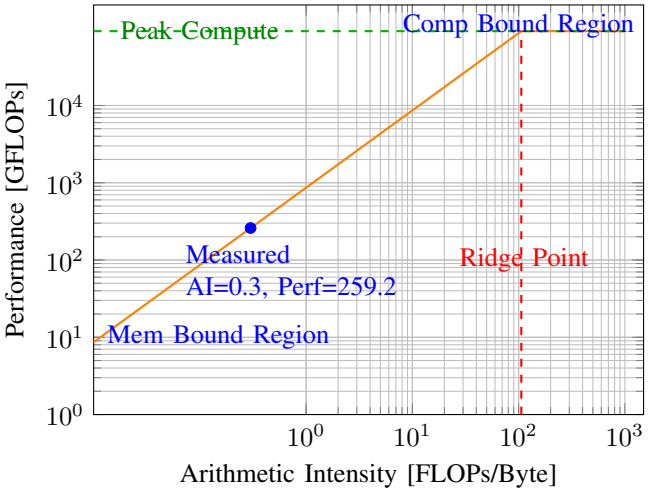

Fig. 16. Roofline model for Algorithm 1 with unroll on NVIDIA L40s

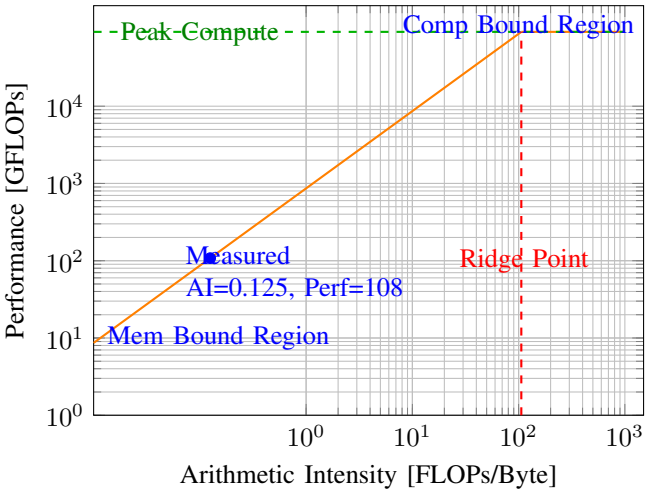

Fig. 17. Roofline model for Algorithm 3 on NVIDIA L40s